

TimescaleDB 완벽 가이드

온라인 게임 서버를 위한

TimescaleDB 완벽 가이드

Game Server
Monitoring
System

C# + SQLKata + Docker로 구현하는 게임 서버 시계열 데이터 시스템

저자: 최흥배 · Claude AI

C# / .NET 9

SQLKata

Docker

20 Chapters

이 책에 대하여

온라인 게임 서버의 시계열 데이터 문제를 TimescaleDB로 해결하는 실전 가이드

무엇을 다루는가

- MMORPG 급 대용량 게임 서버 로그·메트릭 데이터 관리
- TimescaleDB Hypertable, 연속 집계, 압축 실전 활용
- C# + SQLKata + Docker 기반 개발
- 실시간 대시보드·이상 탐지 시스템 구축
- 플레이어 행동 분석 (코호트, 퍼널, 리텐션)

누구를 위한 책인가

- 게임 서버 백엔드 개발자 (주니어~시니어)
- 로그·메트릭 시스템 구축을 처음 경험하는 팀
- MySQL/PostgreSQL로 성능 한계를 느낀 개발자
- C# / .NET 환경에서 시계열 DB 도입을 검토 중인 팀

무엇을 만드는가

- 실시간 서버 성능 메트릭 수집 에이전트
- 대용량 플레이어 이벤트 로그 시스템
- SignalR 기반 실시간 대시보드
- 자동 이상 탐지 및 Slack 알림 시스템
- 자동화된 데이터 압축·보관 파이프라인

전체 목차 — 20개 챕터

6개 파트로 구성된 단계별 학습 로드맵

PART 1 기초

- Ch 1 문제 인식 & TimescaleDB 소개
- Ch 2 개발 환경 구축 (Windows + Docker)

PART 2 핵심 개념

- Ch 3 Hypertable의 이해
- Ch 4 C# + SQLKata CRUD 마스터
- Ch 5 시간 함수 완전 정복

PART 3 데이터 수집

- Ch 6 서버 성능 메트릭 수집 시스템
- Ch 7 게임 로그 수집 및 분석

PART 4 고급 기능

- Ch 8 연속 집계 (Continuous Aggregates)
- Ch 9 데이터 압축 & 보관 정책
- Ch 10 고급 쿼리 최적화

PART 5 운영

- Ch 11 백업 & 복구 전략
- Ch 12 실시간 이상 탐지
- Ch 13 대용량 마이그레이션
- Ch 14 멀티 리전 & 확장성

PART 6 프로젝트

- Ch 15 종합 프로젝트 1 — 실시간 대시보드
- Ch 16 종합 프로젝트 2 — 플레이어 분석
- Ch 17 성능 튜닝 케이스 스터디
- Ch 18 외부 도구 연동 (Grafana 등)
- Ch 19 보안 & 권한 관리
- Ch 20 다음 단계로

PART 1

기초 — 문제 인식 & 환경 구축

Chapter 1 · Chapter 2

왜 TimescaleDB인가? 실전 사례로 배우는 문제의 본질

Ch 1 갑자기 느려진 게임 서버

MySQL 45초 → TimescaleDB 0.3초

실전 위기 사례로 배우는 TimescaleDB

Ch 2 개발 환경 구축

Docker + pgAdmin + Visual Studio

30분 만에 완성하는 개발 환경

Chapter 1: 갑자기 느려진 게임 서버

출시 6개월, 성공한 MMORPG '던전 크로니클'에 닥친 재앙

✓ 출시 1개월

동접자: 5,000명

이벤트: 1,000만 건/일

DB 크기: 8 GB

쿼리 응답: 0.05초

⚠ 3개월 후

동접자: 15,000명 ↑ 300%

이벤트: 3,000만 건/일

DB 크기: 120 GB

쿼리 응답: 2.3초 ↑ 4600%

✗ 6개월 후

동접자: 50,000명 ↑ 1000%

이벤트: 1억 건/일

DB 크기: 580 GB

쿼리 응답: 45초+ ↑ 90000%

💡 데이터 50배 증가 → 쿼리 시간 1,560배 증가 | 단순 RDBMS 구조의 한계 명확히 드러남

Chapter 1: TimescaleDB가 해결하는 5가지 핵심 문제

PostgreSQL 위에 올라탄 시계열 특화 확장

자동 파티셔닝 (Hypertable)

시간 기준 자동 파티셔닝
단 한 줄: `create_hypertable()`
Chunk 단위 자동 관리

Chunk Pruning

7일치 쿼리 → 해당 Chunk만 스캔
MySQL 45초 → TimescaleDB 0.3초
약 150배 빠름

자동 데이터 압축

580 GB → 58 GB (10:1 압축)
스토리지 비용 90% 절감
오래된 Chunk 자동 압축 정책

연속 집계 (Continuous Agg.)

6억 건 집계 → 720건 집계 뷰
0.1초 대시보드 응답
5분마다 자동 갱신

데이터 보관 정책

90일 경과 데이터 자동 삭제
위험한 DELETE 불필요
단 한 줄 정책 설정

Chapter 2: 개발 환경 구축

30분 만에 완성하는 TimescaleDB 개발 환경 — Windows + Docker

01

Docker Desktop

WSL 2 기반
TimescaleDB 컨테이너 실행

02

TimescaleDB

timescale/timescaledb
:latest-pg16 이미지

03

pgAdmin 4

GUI 관리 도구
Query Tool, ERD 뷰어

04

Visual Studio

.NET 9 콘솔 앱
프로젝트 구조 설계

05

NuGet 패키지

Npgsql
SqlKata + Execution

Hello TimescaleDB! — 첫 연결 성공 시 출력

✓ TimescaleDB 버전: 2.13.0 ✓ 현재 시간: 2025-05-18 12:34:56 UTC ✓ 메시지: Hello TimescaleDB!

PART 2

핵심 개념 — Hypertable·CRUD·시간 함수

Chapter 3 · Chapter 4 · Chapter 5

TimescaleDB를 다루는 개발자의 핵심 무기 세 가지

Ch 3 Hypertable의 이해

자동 파티셔닝 & Chunk Pruning
28초 → 0.4초 조회 성능

Ch 4 C# + SQLKata CRUD

타입 안전한 Fluent API
배치 INSERT 16배 성능

Ch 5 시간 함수 완전 정복

time_bucket() 집계 마스터
타임존 & first/last 함수

Chapter 3: Hypertable — 시간을 다루는 마법

자동 파티셔닝으로 수천만 건도 0.4초 안에 조회

Hypertable 구조

Chunk 1: Dec 22~28

Chunk 2: Dec 15~21

Chunk 3: Dec 08~14

Chunk 4: Dec 01~07

↑ 자동 생성 · 자동 관리

단 한 줄로 변환

```
SELECT create_hypertable('player_login_logs',  
    'login_time');
```

Chunk Pruning — 필요한 Chunk만 스캔

5,400만 건 → 해당 주 300만 건만 읽음
28초 → 0.4초 (약 70배 향상)

자동 Chunk 생성 & 크기 조정

```
-- 1일 Chunk 간격 설정  
SELECT set_chunk_time_interval(  
    'player_login_logs',  
    INTERVAL '1 day');
```

Chapter 4 & 5: SQLKata CRUD + 시간 함수

타입 안전한 쿼리 빌더와 time_bucket의 강력한 조합

SQLKata — Fluent API

```
// 실패한 로그인 조회
var results = await db
    .Query("player_login_logs")
    .Where("login_time", ">=", cutoff)
    .Where("login_result", "FAILED")
    .WhereIn("device_type",
        new[] { "Android", "iOS" })
    .OrderByDesc("login_time")
    .GetAsync<PlayerLoginLog>();

// 배치 INSERT (500건 → 150ms)
await db.Query("player_login_logs")
    .InsertAsync(batchData);
```

time_bucket — 시간대별 집계

```
// 5분 단위 동시 접속자
var stats = await db
    .Query("player_login_logs")
    .WhereBetween("login_time",
        startTime, endTime)
    .SelectRaw(
        "time_bucket('5 minutes',"+
        " login_time) as bucket")
    .SelectRaw(
        "COUNT(DISTINCT player_id)" +
        " as ccu")
    .GroupByRaw("bucket")
    .OrderByRaw("bucket")
    .GetAsync();

// DAU, ARPU, 매출 추이도 동일 패턴
```

PART 3

데이터 수집 — 메트릭 & 게임 로그

Chapter 6 · Chapter 7

초당 수천 건 데이터를 효율적으로 저장하는 고성능 수집 파이프라인 구축

Ch 6 서버 성능 메트릭 수집

CPU·메모리·네트워크 1초 주기
1,000건 배치 삽입으로 부하 최소화

Ch 7 게임 로그 수집 & 분석

로그인·아이템·던전·결제 이벤트
JSONB 유연한 컨텍스트 저장

Chapter 6 & 7: 서버 메트릭 수집 + 게임 로그 시스템

실시간 성능 모니터링 Agent와 이벤트 로그 파이프라인



Ch6 — 서버 성능 메트릭 수집

- 수집 항목: CPU, 메모리, 네트워크, 디스크 I/O
- 주기: 1초마다 Windows 성능 카운터 수집
- 배치 삽입: 1,000건 단위 → 데이터베이스 부하 최소화
- 트랜잭션 처리로 데이터 무결성 보장
- server_metrics Hypertable에 자동 파티셔닝 저장
- 실시간 CPU 급증 · 메모리 누수 감지 지원



Ch7 — 게임 로그 수집 & 분석

- 로그인/로그아웃 · 아이템 획득/사용/거래
- 던전 입장/클리어 · 몬스터 처치 · 퀘스트
- JSONB로 유연한 추가 컨텍스트 저장
- 플레이어 행동 패턴 분석 쿼리
- 아이템 경제 흐름 추적 (골드 인플레이션 감지)
- 다중 계정 의심 IP 자동 탐지 (어뷰징 방지)

PART 4

고급 기능 — 집계·압축·최적화

Chapter 8 · Chapter 9 · Chapter 10

실시간 대시보드, 스토리지 비용 90% 절감, 수억 건에서 1초 쿼리

Ch 8 연속 집계

45초 → 0.1초 대시보드
5분 자동 갱신 정책

Ch 9 데이터 압축 & 보관

580 GB → 58 GB (90% 절감)
자동 압축·삭제 파이프라인

Ch 10 고급 쿼리 최적화

EXPLAIN ANALYZE 실전 활용
Chunk Exclusion & 복합 인덱스

Chapter 8: 연속 집계 (Continuous Aggregates)

실시간 대시보드의 비밀 — 45초 → 0.1초

원본 Hypertable → Background Job 자동 집계 → Materialized View → 대시보드 API

✗ 집계 전 — 원본 테이블 직접 조회

- 6억 건 스캔 후 집계
- 쿼리 실행 시간: 45초
- 대시보드 로딩 불가 (타임아웃)
- DB 부하 급증 → 게임 서버 영향

✓ 연속 집계 뷰 조회

- 미리 집계된 720건만 읽음
- 쿼리 실행 시간: 0.1초
- 5분마다 자동 갱신 (정책 설정)
- DB 부하 없이 실시간성 유지

Chapter 9 & 10: 데이터 압축 + 고급 쿼리 최적화

스토리지 90% 절감 & EXPLAIN ANALYZE 실전 활용

✚ Ch9 — 데이터 압축 & 보관 정책

압축 효과 비교

압축 전: 580 GB

압축 후: 58 GB (90% 절감)

- 컬럼 기반 압축 + 델타 인코딩
- 7일 경과 Chunk 자동 압축 정책
- 90일 경과 데이터 자동 삭제 정책
- 압축 상태에서도 정상 쿼리 가능
- 단 2줄로 설정 완료

⚡ Ch10 — 고급 쿼리 최적화

■ EXPLAIN ANALYZE

실행 계획 분석으로 병목 식별
예상·실제 비용 비교

■ Chunk Exclusion 극대화

WHERE 절에 정확한 시간 범위
→ 필요 Chunk만 스캔

■ 복합 인덱스 전략

(player_id, login_time DESC)
자주 쓰는 조합 패턴으로 설계

■ 파티션 프루닝

수억 건 → 1초 이내 쿼리
연속 집계와 병행 활용

PART 5

운영 — 백업·탐지·마이그레이션·확장

Chapter 11 · 12 · 13 · 14

라이브 서비스를 안전하게 운영하기 위한 4가지 필수 역량

Ch 11

백업 & 복구

PITR 30분 복구

Ch 12

이상 탐지

Slack 자동 알림

Ch 13

마이그레이션

무중단 이전 전략

Ch 14

멀티 리전

스트리밍 복제

Chapter 11~14: 백업·이상탐지·마이그레이션·멀티리전

라이브 게임 서버 운영에 필수적인 4가지 시스템

Ch11 백업 & 복구 전략

pg_dump 전체 백업 / WAL 아카이빙
Point-in-Time Recovery (PITR)
30분 안에 서비스 완전 복구
백업 자동화 스크립트 구현

Ch12 실시간 이상 탐지

정적·동적·변화율 임계값 세 가지 패턴
CPU 90% 초과 → Slack 알림 즉시
접속자 급감 → Discord 긴급 알림
이동 평균 기반 오탐 최소화

Ch13 대용량 마이그레이션

MySQL / MongoDB → TimescaleDB
무중단 이전: 쌍방 쓰기 패턴 적용
초기 벌크 로드 후 증분 동기화
전환 검증 체크리스트 포함

Ch14 멀티 리전 & 확장성

스트리밍 복제 (Primary → Standby)
읽기 부하 분산 (Read Replica)
리전별 레이턴시 최소화 전략
Docker 기반 클러스터 구성 실습

PART 6

종합 프로젝트 & 실전 활용

Chapter 15 · 16 · 17 · 18 · 19 · 20

배운 것을 모두 연결하여 실제 운영 가능한 완성 시스템을 구축한다

Ch 15 실시간 대시보드

SignalR + Chart.js

Ch 16 플레이어 행동 분석

코호트·퍼널·리텐션

Ch 17 성능 튜닝

케이스 스터디 실전

Ch 18 외부 도구 연동

Grafana · Kafka · Redis

Ch 19 보안 & 권한 관리

RLS · SSL · 감사 로그

Ch 20 다음 단계로

프로덕션 체크리스트

Chapter 15~20: 종합 프로젝트 & 실전 마스터

이론에서 운영까지 — 완전한 모니터링 시스템 완성

Ch15

실시간 게임 서버 대시보드

- ASP.NET Core Web API 백엔드
- SignalR 실시간 업데이트
- Chart.js 시각화 통합
- SQLKata 최적화 쿼리

Ch18 외부 도구 연동

Grafana · Prometheus · Redis · Kafka + TimescaleDB 통합 파이프라인

Ch16

플레이어 행동 분석 시스템

- 코호트 분석 (D1/D7/D30 리텐션)
- 퍼널 분석 — 이탈 지점 찾기
- 플레이어 세그멘테이션
- ARPU · 전환율 자동 계산

Ch19 보안 & 권한 관리

역할 기반 접근 제어 · SSL/TLS · Row Level Security · 감사 로그

Ch17

성능 튜닝 케이스 스터디

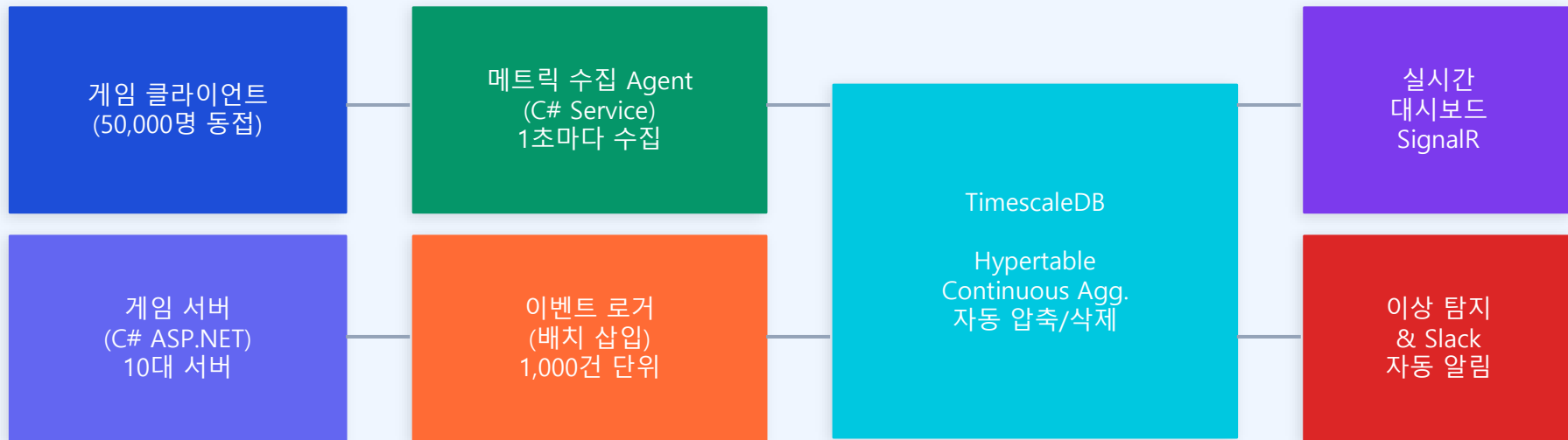
- 대시보드 급격한 성능 저하 대응
- 연속 집계 갱신 지연 해결
- 실제 쿼리 EXPLAIN 분석 실습
- 인덱스 설계 패턴 비교

Ch20 다음 단계로

전체 내용 정리 · 프로덕션 체크리스트 · 지속 성장 학습 로드맵

완성된 시스템 아키텍처

이 책을 마치면 갖게 되는 실전 GameMonitoring 솔루션



이 책을 마치면

- ✓ 초당 수천 건 데이터를 효율적으로 저장
- ✓ 수억 건에서 빠른 조회 실현
- ✓ 실시간 모니터링 대시보드 독립 구축
- ✓ 스토리지 비용 90% 절감
- ✓ 자동화된 이상 탐지 & 알림 시스템
- ✓ 플레이어 행동 분석 인사이트 도출

"데이터는 새로운 석유다.

하지만 정제하지 않으면 쓸모없다."

이제 TimescaleDB로 데이터를 정제하고,
가치 있는 인사이트를 게임에 반영하세요.

■ 온라인 게임 서버를 위한 TimescaleDB 완벽 가이드

저자: 최흥배 · Claude AI

20 Chapters | C# · SQLKata · Docker · Windows 11

TimescaleDB

.NET 9

SQLKata

Docker

SignalR

pgAdmin